

Assignment 1: Converter and Pixel System

CS 240

GitHub Link:

<https://github.com/SeanMValenzuela/CS240.git>

Overview:

In Assignment 1, we were tasked with programming and testing four tasks: building an ASCII-to-decimal converter, building a number-base converter supporting binary, decimal, octal, and hexadecimal values, writing a program that reads an image and prints its pixel values, and writing a program that takes in pixel values and converts it into an image.

ASCII-to-Decimal Converter: When the ASCII-to-decimal converter is initially run, the terminal first asks the user what they want to convert from ASCII to decimal, whether it is a letter, number, sentence, phrase, etc. The converter will then proceed to give out the decimal values of each character line-by-line in the given input.

Number-Base Converter: With the number-base converter, the terminal first requests the user what base they want to convert from, whether it is binary, decimal, octal, or hexadecimal. Similarly in the next step, the terminal asks the user what base they want to convert to. Following that, the user will then input the actual number they want to convert between number-bases and the program will then output the number in the converted form. If the user enters an invalid base, an `IOException` will occur, and if the user enters an invalid number for the selected base, a `NumberFormatException` will occur.

Image-to-Pixel Converter: When run, the image-to-pixel converter begins by asking the user to input the file name of the image they want to pull the pixels from. An important thing to note is that before proceeding, the user should first change the directory into the folder that their desired image is located in. The program will then read the image and write the hex code of every single pixel in the image in a newly created output .txt file.

Pixel-to-Image Converter: The pixel-to-image converter essentially runs the opposite way of the image-to-pixel converter. The terminal first asks the user to input their desired width of the output image, and then immediately after, their desired height as well. The program also prompts the user for the name of the existing .txt file containing all of the hexcode values being translated into an image and as well as the desired filename for the output .png image. When finished, the program will produce a .png image as specified by the user and input .txt file.

Source Code:

ASCII-to-Decimal Converter:

```
import java.util.Scanner;

public class asciiToDecimalConverterHw1 {

    public static void main(String[] args) {
        // Build an ASCII-to-decimal converter.
        Scanner scnr = new Scanner(System.in);
        System.out.println("ASCII-to-decimal converter: Enter what you want to convert.");
        String s1 = scnr.nextLine();

        for (Character c : s1.toCharArray()) {
            System.out.println((int)c);
        }
    }
}
```

Number-Base Converter:

```
import java.util.Scanner;

public class numberBaseConverterHw1 {

    public static void main(String[] args) {
        // Build a number-base converter supporting binary, decimal, octal, and
        hexadecimal.

        Scanner scnr = new Scanner(System.in);
        System.out.println("Number-base converter: Enter the base you want to convert
from (binary, decimal, octal, hexadecimal)");
        String fromType = scnr.nextLine();
        System.out.println("Number-base converter: Enter the type you want to convert
to (binary, decimal, octal, hexadecimal)");
        String toType = scnr.nextLine();
        System.out.println("Number-base converter: Enter the number you want to
convert");
        String toConvert = scnr.nextLine();

        if (fromType.equalsIgnoreCase("binary") || fromType.equalsIgnoreCase("decimal")
|| fromType.equalsIgnoreCase("octal") || fromType.equalsIgnoreCase("hexadecimal")) {
            if (toType.equalsIgnoreCase("binary") || toType.equalsIgnoreCase("decimal")
|| toType.equalsIgnoreCase("octal") || toType.equalsIgnoreCase("hexadecimal")) {
                try {
                    if (fromType.equalsIgnoreCase(toType)) {
                        System.out.println(toConvert);
                    } else if (fromType.equalsIgnoreCase("binary") &&
toType.equalsIgnoreCase("decimal")) {
                        long binDec = Long.parseLong(toConvert, 2);
                        System.out.println(binDec);
                    }
                }
            }
        }
    }
}
```

```

        } else if (fromType.equalsIgnoreCase("binary") &&
toType.equalsIgnoreCase("octal")) {
            long output = Long.parseLong(toConvert, 2);
            String binOct = Long.toOctalString(output);
            System.out.println(binOct);
        } else if (fromType.equalsIgnoreCase("binary") &&
toType.equalsIgnoreCase("hexadecimal")) {
            long output = Long.parseLong(toConvert, 2);
            String binHex = Long.toHexString(output);
            System.out.println(binHex);

        } else if (fromType.equalsIgnoreCase("decimal") &&
toType.equalsIgnoreCase("binary")) {
            long output = Long.parseLong(toConvert, 10);
            if (output < 0) {
                output = output & 0xFFFFFFFFL;
            }
            String decBin = Long.toBinaryString(output);
            System.out.println(decBin);
        } else if (fromType.equalsIgnoreCase("decimal") &&
toType.equalsIgnoreCase("octal")) {
            long output = Long.parseLong(toConvert, 10);
            if (output < 0) {
                output = output & 0xFFFFFFFFL;
            }
            String decOct = Long.toOctalString(output);
            System.out.println(decOct);
        } else if (fromType.equalsIgnoreCase("decimal") &&
toType.equalsIgnoreCase("hexadecimal")) {
            long output = Long.parseLong(toConvert, 10);
            if (output < 0) {
                output = output & 0xFFFFFFFFL;
            }
            String decHex = Long.toHexString(output);
            System.out.println(decHex);

        } else if (fromType.equalsIgnoreCase("octal") &&
toType.equalsIgnoreCase("binary")) {
            long output = Long.parseLong(toConvert, 8);
            String octBin = Long.toBinaryString(output);
            System.out.println(octBin);

```

```

        } else if (fromType.equalsIgnoreCase("octal") &&
toType.equalsIgnoreCase("decimal")) {
            long octDec = Long.parseLong(toConvert, 8);
            System.out.println(octDec);
        } else if (fromType.equalsIgnoreCase("octal") &&
toType.equalsIgnoreCase("hexadecimal")) {
            long output = Long.parseLong(toConvert, 8);
            String octHex = Long.toHexString(output);
            System.out.println(octHex);

        } else if (fromType.equalsIgnoreCase("hexadecimal") &&
toType.equalsIgnoreCase("binary")) {
            long output = Long.parseLong(toConvert, 16);
            String hexBin = Long.toBinaryString(output);
            System.out.println(hexBin);
        } else if (fromType.equalsIgnoreCase("hexadecimal") &&
toType.equalsIgnoreCase("decimal")) {
            long hexDec = Long.parseLong(toConvert, 16);
            System.out.println(hexDec);
        } else if (fromType.equalsIgnoreCase("hexadecimal") &&
toType.equalsIgnoreCase("octal")) {
            long output = Long.parseLong(toConvert, 16);
            String hexOct = Long.toOctalString(output);
            System.out.println(hexOct);
        } else {
            System.out.println("Invalid base. Please enter binary, decimal,
octal, or hexadecimal.");
        }
    } catch (NumberFormatException e) {
        System.out.println("Invalid number for the selected base");
    }
} else {
    System.out.println("Invalid base. Enter binary, decimal, octal, or
hexadecimal");
}
} else {
    System.out.println("Invalid base. Enter binary, decimal, octal, or
hexadecimal");
}
}
}
}

```

Image-to-Pixel Converter:

```
import java.io.File;
import java.io.IOException;
import java.util.Scanner;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;
import java.io.PrintWriter;

public class imagePixelConverterHwl {
    public static void main(String[] args) {
        // Write a program that reads an image and prints its pixel values.
        Scanner scnr = new Scanner(System.in);
        try {
            System.out.println("Image to pixel converter: Enter filename of desired image you want to convert");
            String fileName = scnr.next();
            File imageFile = new File(fileName);
            System.out.println("Image to pixel converter: Enter desired filename of .txt output file (include .txt at the end)");
            String outputName = scnr.next();
            PrintWriter writer = new PrintWriter(outputName);
            BufferedImage image = ImageIO.read(imageFile);
            int imgHeight = image.getHeight();
            int imgWidth = image.getWidth();
            for (int i = 0; i < imgWidth; i++) {
                for (int j = 0; j < imgHeight; j++) {
                    int pixelColor = image.getRGB(i, j);
                    String hexColor = String.format("%08X", pixelColor);
                    writer.print(hexColor);
                    writer.print(" ");
                }
                writer.print("\n");
            }
            writer.close();
        } catch (IOException e) {
            System.out.println("Unable to read file. Try again.");
        }
    }
}
```

Pixel-to-Image Converter:

```
import java.util.Scanner;
import java.io.File;
```

```

import java.io.IOException;
import java.io.FileNotFoundException;
import java.awt.image.BufferedImage;
import javax.imageio.ImageIO;

public class pixelImageConverterHwl {
    public static void main(String[] args) {
        // Write a program that consumes pixel values and creates an image.
        Scanner scnr = new Scanner(System.in);
        System.out.println("Pixel to image converter: Enter desired image width");
        int width = scnr.nextInt();
        System.out.println("Pixel to image converter: Enter desired image height");
        int height = scnr.nextInt();
        System.out.println("Pixel to image converter: Enter filename of .txt file
containing hexcode values (include .txt)");
        String inputFileName = scnr.next();
        File inputFile = new File(inputFileName);
        System.out.println("Pixel to image converter: Enter desired output file name
(include .png)");
        String outputFileName = scnr.next();
        File outputFile = new File(outputFileName);
        BufferedImage outputImage = new BufferedImage(width, height,
BufferedImage.TYPE_INT_ARGB);
        try {
            Scanner fileScnr = new Scanner(inputFile);
            for (int i = 0; i < width; i++) {
                for (int j = 0; j < height; j++) {
                    String pixelColor = fileScnr.next();
                    int rgb = Integer.parseUnsignedInt(pixelColor, 16);
                    outputImage.setRGB(i, j, rgb);
                }
            }
            fileScnr.close();
            ImageIO.write(outputImage, "png", outputFile);
        } catch (FileNotFoundException e) {
            System.out.println("File cannot be found. Please try again.");
        } catch (IOException e) {
            System.out.println("Cannot create image file.");
        }
    }
}

```

README.md:

Assignment 1: Converter and Pixel System (CS 240)

Overview

In Assignment 1, we were tasked with programming and testing four tasks: building an ASCII-to-decimal converter, building a number-base converter supporting binary, decimal, octal, and hexadecimal values, writing a program that reads an image and prints its pixel values, and writing a program that takes in pixel values and converts it into an image.

ASCII-to-Decimal Converter

When the ASCII-to-decimal converter is initially run, the terminal first asks the user what they want to convert from ASCII to decimal, whether it is a letter, number, sentence, phrase, etc. The converter will then proceed to give out the decimal values of each character line-by-line in the given input.

Number-Base Converter

With the number-base converter, the terminal first requests the user what base they want to convert from, whether it is binary, decimal, octal, or hexadecimal. Similarly in the next step, the terminal asks the user what base they want to convert to. Following that, the user will then input the actual number they want to convert between number-bases and the program will then output the number in the converted form. If the user enters an invalid base, an `IOException` will occur, and if the user enters an invalid number for the selected base, a `NumberFormatException` will occur.

Image-to-Pixel Converter

When run, the image-to-pixel converter begins by asking the user to input the file name of the image they want to pull the pixels from. An important thing to note is that before proceeding, the user should first change the directory into the folder that their desired image is located in. The program will then read the image and write the hex code of every single pixel in the image in a newly created output `.txt` file.

Pixel-to-Image Converter

The pixel-to-image converter essentially runs the opposite way of the image-to-pixel converter. The terminal first asks the user to input their desired width of the output image, and then immediately after, their desired height as well. The program also prompts the user for the name of the existing `.txt` file containing all of the hexcode values being translated into an image and as

well as the desired filename for the output .png image. When finished, the program will produce a .png image as specified by the user and input .txt file.

Test Evidence/Screenshots/Generated Images

ASCII-to-Decimal Converter:

<p align="left">
</p>

Number-Base Converter:

Binary to Decimal:

<p align="left">
</p>

Hexadecimal to Octal:

<p align="left">
</p>

Zero (Octal to Decimal):

<p align="left">
</p>

Largest Supported Unsigned Value (4294967295, Decimal to Binary):

<p align="left">
</p>

Negative Two's-Complement Value (-1, Decimal to Hexadecimal):

<p align="left">
</p>

Image-to-Pixel Converter:

<p align="left">
</p>

Input Image (blue.png (5x5)):

<p align="left">
</p>

Output File (bluepixels.txt):

<p align="left">
</p>

Pixel-to-Image Converter:

<p align="left">
</p>

Input File (bluepixels.txt):

<p align="left">
</p>

Output Image (bluenew.png (5x5)):

<p align="left">
</p>

AI Acknowledgement

- Used ChatGPT-5.6 Luna to understand how to modify code for number-base converter to support the largest supported 32-bit unsigned value of 4294967295, and representing negative values using 32-bit two's-complement
- Used ChatGPT-5.6 Luna to learn how to change/format ARGB color values to hexadecimal
- Used ChatGPT-5.6 Luna for assistance in debugging, developing and identifying appropriate test cases for the program

Test Evidence/Screenshots/Generated Images:

ASCII-to-Decimal Converter:

```
ASCII-to-decimal converter: Enter what you want to convert.  
Hello World!  
72  
101  
108  
108  
111  
32  
87  
111  
114  
108  
100  
33
```

Number-Base Converter:

Binary to Decimal:

```
Number-base converter: Enter the base you want to convert from (binary, decimal, octal, hexadecimal)  
binary  
Number-base converter: Enter the type you want to convert to (binary, decimal, octal, hexadecimal)  
decimal  
Number-base converter: Enter the number you want to convert  
1111  
15
```

Hexadecimal to Octal:

```
Number-base converter: Enter the base you want to convert from (binary, decimal, octal, hexadecimal)  
hexadecimal  
Number-base converter: Enter the type you want to convert to (binary, decimal, octal, hexadecimal)  
octal  
Number-base converter: Enter the number you want to convert  
B3AD  
131655
```

Zero (Octal to Decimal):

```
Number-base converter: Enter the base you want to convert from (binary, decimal, octal, hexadecimal)  
octal  
Number-base converter: Enter the type you want to convert to (binary, decimal, octal, hexadecimal)  
decimal  
Number-base converter: Enter the number you want to convert  
0  
0
```

Largest Supported Unsigned Value (4294967295, Decimal to Binary):

```
Number-base converter: Enter the base you want to convert from (binary, decimal, octal, hexadecimal)  
decimal  
Number-base converter: Enter the type you want to convert to (binary, decimal, octal, hexadecimal)  
binary  
Number-base converter: Enter the number you want to convert  
4294967295  
11111111111111111111111111111111
```

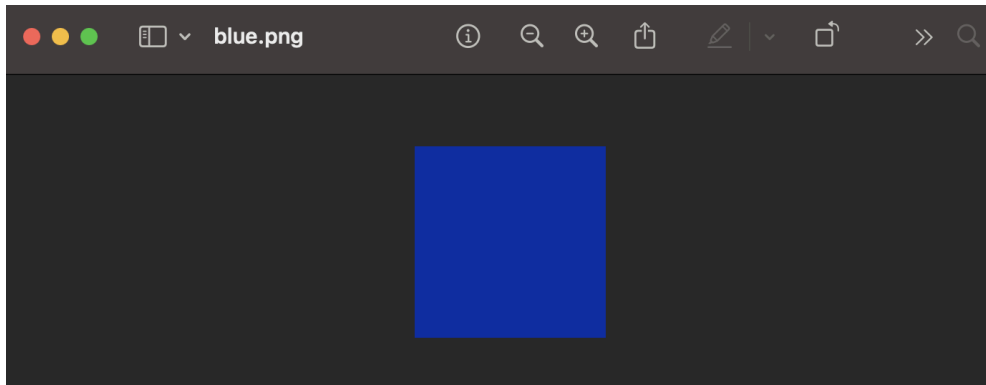
Negative Two's-Complement Value (-1, Decimal to Hexadecimal):

```
Number-base converter: Enter the base you want to convert from (binary, decimal, octal, hexadecimal)
decimal
Number-base converter: Enter the type you want to convert to (binary, decimal, octal, hexadecimal)
hexadecimal
Number-base converter: Enter the number you want to convert
-1
ffffffff
```

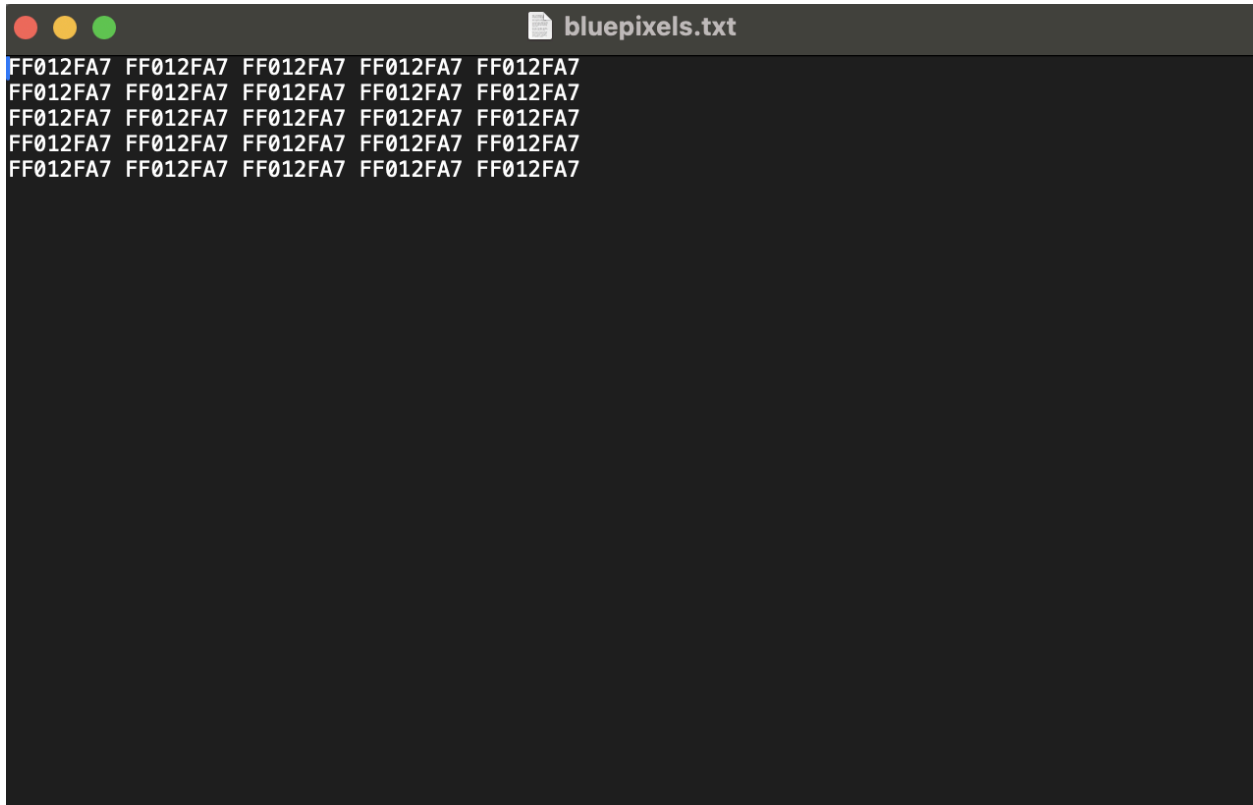
Image-to-Pixel Converter:

```
Image to pixel converter: Enter filename of desired image you want to convert
blue.png
Image to pixel converter: Enter desired filename of .txt output file (include .txt at the end)
bluepixels.txt
○ seanvalenzuela@Seans-MacBook-Air-2 CS240 %
```

Input Image (blue.png (5x5)):



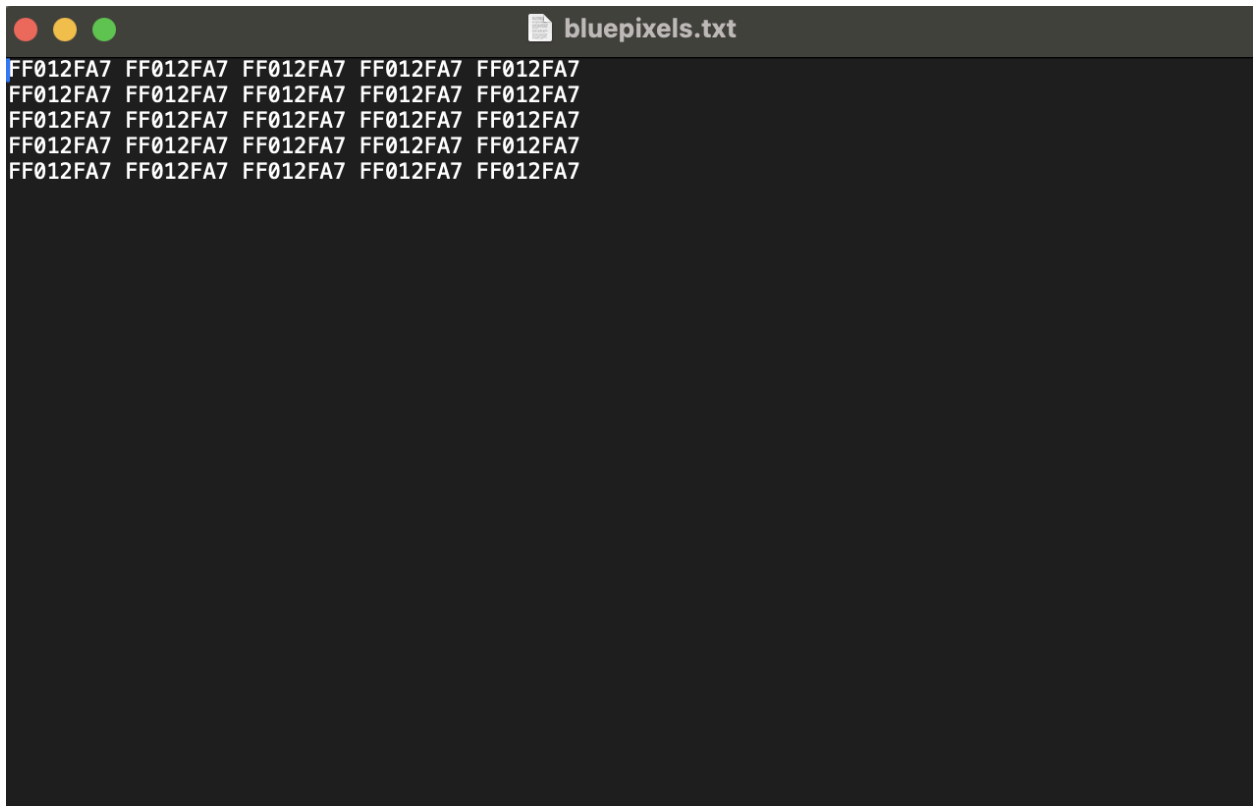
Output File (bluepixels.txt):



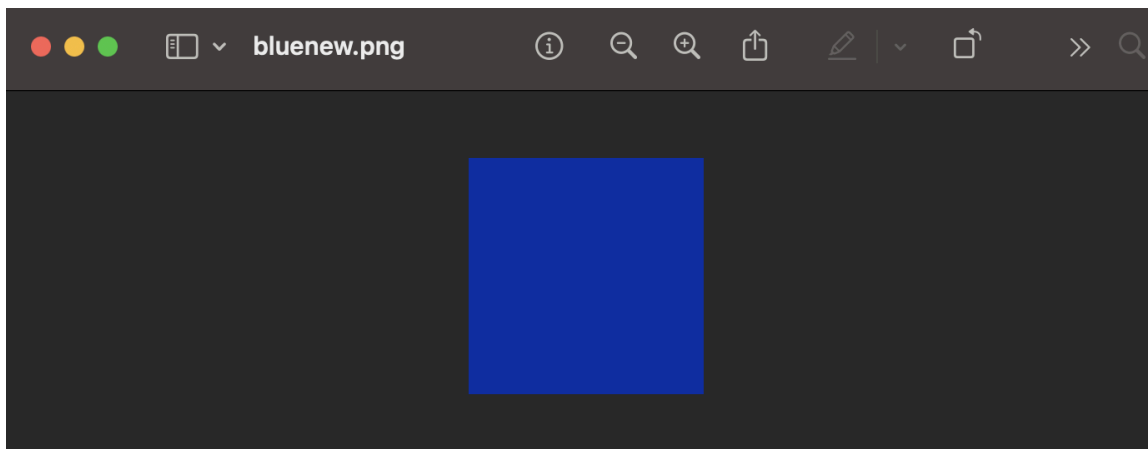
Pixel-to-Image Converter:

```
Pixel to image converter: Enter desired image width
5
Pixel to image converter: Enter desired image height
5
Pixel to image converter: Enter filename of .txt file containing hexcode values (include .txt)
bluepixels.txt
Pixel to image converter: Enter desired output file name (include .png)
bluenew.png
```

Input File (bluepixels.txt):



Output Image (bluenew.png (5x5)):



AI Acknowledgement:

- Used ChatGPT-5.6 Luna to understand how to modify code for number-base converter to support the largest supported 32-bit unsigned value of 4294967295, and representing negative values using 32-bit two's-complement
- Used ChatGPT-5.6 Luna to learn how to change/format ARGB color values to hexadecimal
- Used ChatGPT-5.6 Luna for assistance in debugging, developing and identifying appropriate test cases for the program